

1. Introduction

This report aims to compare the performance of a range of classification algorithms with the same input dataset in numeric and nominal forms, and to discuss the insights gained from the comparison. The report expands its discussion to different aspects of the algorithms as the dataset's characteristics cause the results to be unexpected. It implicitly shows the necessity to be aware of these characteristics and address them to produce more powerful classifier.

2. Data

The dataset used throughout this study, the Pima Indian diabetes dataset, contains 768 instances, where each of which is described by 8 attributes corresponding to a patient's record. The ninth column is the classification of the instance, indicating whether the patient was tested positive for diabetes or not. Numerical and nominal version of same data were provided in two distinct files. 500 instances of the dataset are classified as "no" and the other 268 instances are classified as "yes".

3. DT* Algorithm

The DT* algorithm was created by an addition of a pre-pruning step to the DT algorithm. It adds an extra stopping criteria to the existing DT building step. It prevents a node to be an internal node, in other words, to be split, if it has too few samples remaining.

The standard DT algorithm was modified to check the additional criteria while building the tree. After checking the case c (list of examples is empty), DT* checks if the size of the list is below the threshold (< 77 examples = ~10% of total examples). If it is below the threshold, the current subtree (node) becomes a leaf node. Then it checks other stopping criteria as the standard DT* does.

The main motivation to this modification was to avoid overfitting. The standard algorithm had likeliness to occur overfitting as it would not prevent splitting where a split is not meaningful enough. If the number of examples of the current subtree is too small, it is likely to create child nodes with the number of examples not enough to be a meaningful split and likely to contribute to overfitting.

4. Results and discussion

Numeric Data	ZeroR	1R	1NN	5NN	NB	MLP	SVM	MyNB
Accuracy*	0.651	0.708	0.678	0.745	0.751	0.754	0.763	0.745
Recall (yes)	0.000	0.474	0.545	0.601	0.604	0.657	0.530	0.601
Recall (no)	1.000	0.834	0.750	0.822	0.830	0.806	0.888	0.822

Table 4.1 Useful measures of cross-validation results of classification algorithms for numeric data

*: (Number of correctly classified instances) / (Number of all instances)

Nominal Data	DT-unpruned	DT-pruned	MyDT	MyDT*	Bagg	Boost	RF
Accuracy*	0.750	0.754	0.732	0.784	0.749	0.762	0.732
Recall (yes)	0.586	0.619	0.623	0.649	0.604	0.642	0.571
Recall (no)	0.838	0.826	0.790	0.856	0.826	0.826	0.818

Table 4.2 Useful measures of cross-validation results of classification algorithms for nominal data

*: (Number of correctly classified instances) / (Number of all instances)

J48 unpruned tree

gc = high

```

| bmi = high
| | sft = high
| | | tp = low
| | | | dpf = high
| | | | | age = high: yes (16.0/5.0)
| | | | | age = low
| | | | | bp = high: yes (11.0/5.0)
| | | | | bp = low: no (5.0/2.0)
| | | | dpf = low
| | | | | bp = high: no (43.0/19.0)
| | | | | bp = low: yes (10.0/4.0)
| | | tp = high
| | | | bp = high: yes (29.0/8.0)
| | | | bp = low
| | | | | dpf = high: no (2.0)
| | | | | dpf = low: yes (3.0)
| | sft = low: no (13.0/4.0)
| bmi = low: no (29.0/4.0)

```

gc = low

```

| bmi = high
| | si = high
| | | age = high
| | | | dpf = high: yes (7.0/3.0)
| | | | dpf = low: no (28.0/4.0)

```

- | | | age = low: no (43.0/4.0)
- | | si = low: no (48.0/2.0)
- | bmi = low: no (66.0)
- gc = very high
- | si = high
- | | bmi = high: yes (103.0/16.0)
- | | bmi = low
- | | | age = high: yes (12.0/3.0)
- | | | age = low: no (4.0/1.0)
- | si = low: no (3.0/1.0)
- gc = medium
- | age = high
- | | si = high
- | | | bmi = high
- | | | | dpf = high: yes (37.0/10.0)
- | | | | dpf = low
- | | | | | bp = high: no (57.0/24.0)
- | | | | | bp = low
- | | | | | sft = high: yes (15.0/7.0)
- | | | | | sft = low: no (3.0/1.0)
- | | | bmi = low: no (27.0/3.0)
- | | si = low: no (8.0)
- | age = low
- | | bmi = high
- | | | tp = low
- | | | | sft = high
- | | | | | dpf = high
- | | | | | bp = high: no (17.0/2.0)
- | | | | | bp = low: yes (7.0/3.0)
- | | | | | dpf = low: no (54.0/8.0)
- | | | | sft = low: no (24.0/1.0)
- | | | tp = high: yes (2.0/1.0)
- | | bmi = low: no (42.0/1.0)

J48 pruned tree

gc = high

| bmi = high

| | sft = high: yes (119.0/51.0)

| | sft = low: no (13.0/4.0)

| bmi = low: no (29.0/4.0)

gc = low: no (192.0/14.0)

gc = very high: yes (122.0/24.0)

gc = medium

| age = high

| | bmi = high

| | | dpf = high: yes (37.0/10.0)

| | | dpf = low: no (80.0/33.0)

| | bmi = low: no (30.0/3.0)

| age = low: no (146.0/17.0)

DT

--

gc = high

| bmi = high

| | age = high

| | | dpf = high

| | | | bp = high

| | | | | tp = high

| | | | | | sft = high

| | | | | | | si = high: yes

| | | | | | tp = low

| | | | | | sft = high

| | | | | | | si = high: yes

| | | | | | | si = low: yes

| | | | | | sft = low: yes

| | | | | bp = low

| | | | | | tp = high: no

| | | | | | tp = low

| | | | | | sft = high

| | | | | | | si = high: yes

| | | | | dpf = low

| | | | si = high
| | | | | sft = high
| | | | | | bp = high
| | | | | | | tp = high: yes
| | | | | | | tp = low: no
| | | | | | | bp = low
| | | | | | | tp = high: yes
| | | | | | | tp = low: yes
| | | | | sft = low
| | | | | | tp = high: no
| | | | | | tp = low
| | | | | | | bp = high: yes
| | | | | | | bp = low: no
| | | | | si = low: yes
| | | age = low
| | | sft = high
| | | | dpf = high
| | | | | bp = high
| | | | | | tp = low
| | | | | | | si = high: yes
| | | | | | | bp = low
| | | | | | | tp = low
| | | | | | | si = high: no
| | | | | dpf = low
| | | | | | bp = high
| | | | | | | tp = low
| | | | | | | si = high: no
| | | | | | | bp = low
| | | | | | | tp = low
| | | | | | | si = high: yes
| | | | | sft = low
| | | | | bp = high: no
| | | | | bp = low
| | | | | | si = high
| | | | | | | dpf = high
| | | | | | | tp = low: yes

| | | | | dpf = low: no
| | | | | si = low: no
| bmi = low
| | sft = high
| | | si = high
| | | | dpf = high: no
| | | | dpf = low
| | | | | age = high
| | | | | | bp = high
| | | | | | | tp = low: no
| | | | | | | bp = low: no
| | | | | | age = low
| | | | | | | bp = high: no
| | | | | | | bp = low
| | | | | | | | tp = low: yes
| | | | si = low
| | | | | dpf = high: yes
| | | | | dpf = low: no
| | | sft = low: no
gc = medium
| age = high
| | bmi = high
| | | dpf = high
| | | | tp = high: yes
| | | | | tp = low
| | | | | | sft = high
| | | | | | | bp = high
| | | | | | | | si = high: yes
| | | | | | | | bp = low
| | | | | | | | si = high: yes
| | | | | | sft = low: yes
| | | | dpf = low
| | | | | si = high
| | | | | | bp = high
| | | | | | | tp = high
| | | | | | | | sft = high: no

| | | | | | tp = low
| | | | | | sft = high: no
| | | | | | sft = low: yes
| | | | | bp = low
| | | | | | sft = high
| | | | | | tp = high: yes
| | | | | | tp = low: yes
| | | | | | sft = low
| | | | | | tp = low: no
| | | | si = low: no
| | bmi = low
| | | bp = high
| | | | tp = high: no
| | | | tp = low
| | | | dpf = high: no
| | | | dpf = low
| | | | | | sft = high
| | | | | | si = high: no
| | | | | | sft = low: no
| | | bp = low
| | | | tp = high: yes
| | | | tp = low
| | | | | sft = high: no
| | | | | sft = low
| | | | | | si = high
| | | | | | dpf = high: no
| age = low
| | bmi = high
| | | sft = high
| | | | tp = high
| | | | | bp = high
| | | | | | si = high
| | | | | | dpf = low: yes
| | | | | tp = low
| | | | | dpf = high
| | | | | | bp = high

| | | | | | | si = high: no
| | | | | | | si = low: no
| | | | | | | bp = low
| | | | | | | si = high: yes
| | | | | | | si = low: yes
| | | | | | | dpf = low
| | | | | | | bp = high
| | | | | | | si = high: no
| | | | | | | si = low: no
| | | | | | | bp = low
| | | | | | | si = high: no
| | | | | | | si = low: no
| | | | | | | sft = low
| | | | | | | dpf = high
| | | | | | | bp = high: no
| | | | | | | bp = low
| | | | | | | si = high
| | | | | | | tp = low: no
| | | | | | | si = low: no
| | | | | | | dpf = low: no
| | | | | | | bmi = low
| | | | | | | dpf = high
| | | | | | | si = high: no
| | | | | | | si = low
| | | | | | | bp = high: no
| | | | | | | bp = low
| | | | | | | tp = low
| | | | | | | sft = low: yes
| | | | | | | dpf = low: no
gc = low
| | | | | | | bmi = high
| | | | | | | si = high
| | | | | | | age = high
| | | | | | | dpf = high
| | | | | | | bp = high
| | | | | | | tp = high

| | | | | | | sft = high: yes
| | | | | | | sft = low: no
| | | | | | | tp = low
| | | | | | | sft = high: no
| | | | | | | sft = low: yes
| | | | | bp = low: yes
| | | | dpf = low
| | | | | sft = high
| | | | | | tp = high
| | | | | | | bp = high: no
| | | | | | | bp = low: no
| | | | | | | tp = low
| | | | | | | bp = high: no
| | | | | | | bp = low: no
| | | | | sft = low: no
| | | age = low
| | | | bp = high: no
| | | | bp = low
| | | | | sft = high
| | | | | | dpf = high
| | | | | | | tp = low: no
| | | | | | | dpf = low
| | | | | | | tp = low: no
| | | | | sft = low: no
| | si = low
| | | bp = high
| | | | age = high: no
| | | | age = low
| | | | | sft = high
| | | | | | dpf = high: yes
| | | | | | dpf = low
| | | | | | | tp = low: no
| | | | | sft = low: no
| | | bp = low: no
| bmi = low: no
gc = very high

| si = high
| | bmi = high
| | | tp = high
| | | | dpf = high: yes
| | | | dpf = low
| | | | | bp = high
| | | | | | sft = high
| | | | | | | age = high: yes
| | | | | | | bp = low
| | | | | | | sft = high
| | | | | | | age = high: yes
| | | | | | | tp = low
| | | | | | | age = high
| | | | | | | dpf = high
| | | | | | | sft = high
| | | | | | | bp = high: yes
| | | | | | | bp = low: yes
| | | | | | | sft = low: yes
| | | | | | | dpf = low
| | | | | | | bp = high
| | | | | | | sft = high: yes
| | | | | | | sft = low: yes
| | | | | | | bp = low
| | | | | | | sft = high: yes
| | | | | | | sft = low: yes
| | | | | | | age = low
| | | | | | | dpf = high: yes
| | | | | | | dpf = low
| | | | | | | sft = high
| | | | | | | bp = high: yes
| | | | | | | bp = low: yes
| | | | | | | sft = low
| | | | | | | bp = high: yes
| | | | | | | bp = low: no
| | | | | | | bmi = low
| | | | | | | age = high

| | | | sft = high
| | | | | tp = high
| | | | | | dpf = high
| | | | | | | bp = high: yes
| | | | | | | bp = low: yes
| | | | | | dpf = low: yes
| | | | | | tp = low
| | | | | | bp = high
| | | | | | | dpf = low: yes
| | | | | | | bp = low
| | | | | | | dpf = low: yes
| | | | sft = low: yes
| | | age = low
| | | | bp = high
| | | | | sft = high: no
| | | | | sft = low
| | | | | | tp = low
| | | | | | | dpf = low: yes
| | | | bp = low: no
| si = low
| | dpf = high: yes
| | dpf = low: no

DT*

gc = very high

| si = high

| | bmi = high

| | | tp = high: yes

| | | tp = low: yes

| | bmi = low: yes

| si = low: no

gc = high

| bmi = high

| | age = high

| | | dpf = high: yes

```

| | | dpf = low: yes
| | age = low: no
| bmi = low: no
gc = medium
| age = high
| | bmi = high
| | | dpf = high: yes
| | | dpf = low
| | | | si = high: no
| | | | si = low: no
| | bmi = low: no
| age = low
| | bmi = high
| | | sft = high
| | | | tp = high: yes
| | | | tp = low
| | | | | dpf = high: no
| | | | | dpf = low: no
| | | sft = low: no
| | bmi = low: no
gc = low
| bmi = high
| | si = high
| | | age = high: no
| | | age = low: no
| | si = low: no
| bmi = low: no

```

Considering the accuracy of ZeroR algorithm to be baseline for the others, all the other algorithms exceeded the baseline performance. The performance of the classification algorithms except for ZeroR ranges from 0.678 (1NN) to 0.784 (MyDT*). It is important to notice that since ZeroR chose the majority class “no”, every instance was classified as “no”, resulting in 1.000 recall of the “no” class. There is remarkable contrast between the recall of both classes in every classification algorithm in addition to ZeroR. This is due to the imbalance in class distribution of the training set. Throughout the process, the majority class contributed most to build and improve the classification model. Models that prioritize improving accuracy are even more likely to not improve minority class’ recall, since accuracy hides minority performance.

Table 4.2 shows the improvement of measures with a pre-pruning step added to MyDT*, +0.052, +0.026, and +0.066 in accuracy, recall of “yes” and recall of “no” respectively. This implies overall improvement in classification performance of both classes. The pre-pruning step appears to successfully mitigate overfitting. This could also imply that the class is determined by few major attributes. Reduction in the size of the tree is also demonstrated in the diagram above.

There is slight difference between performance of NB and MyNB, 0.006, 0.003 and 0.008 respectively. There could be difference in implementation details. Even though comparison in full detail is impossible, the difference is small enough to argue that both run similarly with negligible performance difference.

There is noticeable contrast between Decision Tree algorithms’ performance. The pruned tree showed slight gain in accuracy and recall (yes) (+0.004 and +0.033 respectively), with slight loss in recall (no) of -0.012. Even though there was a trade-off between the recalls as a result of pruning, it can still be considered meaningful that the pruned tree detected more positives given that it is used to help medical diagnosis. Moreover, the diagrams show the decrease in tree sizes, which directly affect the computational efficiency and can be considered as an important matter in time-sensitive systems.

Difference in implementation details between DTs and MyDTs caused contrast in performance between them. MyDT underperformed DTs in aspect of accuracy, however, it was more capable of detecting positives. In spite of the loss of recall (no), it could still be a reasonable choice in medical domain. MyDT*, on the other hand, outperformed both DTs in significant extent in all measures. This implies that attribute selection criteria and splitting criteria of MyDT* are more suitable for classification of the dataset than DTs’. The tree size was smaller than DT-unpruned and larger than DT-pruned.

The ensembles of trees failed to improve the accuracy significantly than the single-tree algorithms. There are two main limitations that are hindering the ensembles to outperform single-tree algorithms. First, due to the imbalance in class distribution, bagging or forests will mainly vote for the majority, and boosting may overfit noise or ignore the tiny class. Second, unlike numeric data, the ensembles are vulnerable to discrete data in absence of special unseen attribute handling.

5. Conclusion

Every other algorithm classified better than ZeroR, even though the class distribution was skewed. However, there was significant loss in class “yes” recall due to the imbalance. Additional splitting criteria to DT resulted in remarkable performance improvement. While NB and MyNB showed insignificant difference, DTs and MyDTs showed diversity in both performance and produced trees. The ensembles of trees were unable to enhance the single-tree algorithm, affected by the unbalanced and categorical dataset.

It appears to be necessary to address the imbalance in the dataset with adjustments in classification algorithms to classify patients safely enough. Future studies can focus on analysing the performance of optimized algorithms or those with balanced dataset.

6. Reflection

The most important thing that I have learned throughout this assignment is that paying attention to the key characteristics of the dataset and addressing them is essential to build classifiers that will perform better. I did not expect the unbalanced class distribution would prevent all classifiers from detecting positives (yes) in this extent. Also, I expected the ensembles could mitigate the impact and excel in some way. However, the results were not as I expected, and I had an opportunity to question the reason why, during which time I could observe the impact of the imbalance and could examine different aspects of the classifiers. This insight can be applied to future classification tasks.

Appendix

```
import csv
import math
from collections import Counter

# pre-pruning parameters
MIN_SAMPLES = 77      # minimum number of examples to allow a split
ATTRIBUTE_NAMES = ['tp', 'gc', 'bp', 'sft', 'si', 'bmi', 'dpf', 'age']

class Node:
    # cl is class for leaf node, or majority class for internal node
    # only internal node will have non-null attr_index and not empty children
    # children is a dict attribute value : child node
    def __init__(self, cl, attr_index=None, children=None):
        self.cl = cl
        self.attr_index = attr_index
        self.children = children or {}

def load_data(filename, has_class=True):
    data = []
    with open(filename) as f:
        reader = csv.reader(f)
        for row in reader:
```

```

    if has_class:
        # list of attributes and class
        *attrs, cl = row
        data.append((attrs, cl))
    else:
        attrs = row
        data.append(attrs)
return data

```

```

# return majority class of subset (examples)
# cls is list of classes
def mode(cls):
    counts = Counter(cls)
    return 'yes' if counts['yes'] >= counts['no'] else 'no'

```

```

# cls is list of classes
def entropy(cls):

```

```

    output = 0.0

```

```

    if not cls:
        return output

```

```

    counts = Counter(cls)
    for count in counts.values():
        prob = count / len(cls)
        output -= prob * math.log2(prob)
    return output

```

```

# examples is list of (list of attributes, class)
def choose_attribute(attr_indices, examples):

```

```

    # setup

```

```

best_attr = None
best_info_gain = -1
h_examples = entropy([cl for _, cl in examples])

# calculate information gain of each attribute
for attr in attr_indices:

    # subsets is attribute value : list of classes
    subsets = {}
    for attrs, cl in examples:
        subsets.setdefault(attrs[attr], []).append(cl)

    info_gain = h_examples
    for subset in subsets.values():
        info_gain -= len(subset) / len(examples) * entropy(subset)
    if info_gain > best_info_gain:
        best_info_gain = info_gain
        best_attr = attr

return best_attr

# examples is a subset of training data
# attr_indices is a list of attribute indices
# default is expected to be majority class of subset of parent node if specified
def build_dt(examples, attr_indices, default=None):

    # extract list of classes from examples (train_data)
    cls = [cl for _, cl in examples]

    # stopping criterion - stop (return a leaf node) if case a, b or c
    # case c) if examples is empty then return default
    if not examples:
        return Node(default)

    # pre-pruning) too few samples

```



```

if len(examples) < MIN_SAMPLES:
    return Node(mode(cls))

# case a) else if all examples have same classification then return the classification
if all(cl == cls[0] for cl in cls):
    return Node(cls[0])

# case b) else if attr_indices is empty then return mode(cls)
if not attr_indices:
    return Node(mode(cls))

# index of attribute with greatest information gain
best_attr = choose_attribute(attr_indices, examples)

# build tree by recursing
tree = Node(mode(cls), best_attr)
values = set(attrs[best_attr] for attrs, _ in examples)
for v_i in values:
    examples_i = [(attrs, cl) for attrs, cl in examples if attrs[best_attr] == v_i]
    subtree = build_dt(examples_i, [i for i in attr_indices if i != best_attr], mode(cls))
    tree.children[v_i] = subtree

# return root node
return tree

```

```

def dt_to_string(node, depth=0):

```

```

    # leaf node
    if not node.children:
        return f"{node.cl}\n"

    # internal node
    lines = []
    attr_name = ATTRIBUTE_NAMES[node.attr_index]
    for value, child in node.children.items():

```

```

    prefix = "| " * depth
    if child.children:
        # child is internal node
        lines.append(f"{prefix}{attr_name} = {value}\n")
        lines.append(dt_to_string(child, depth + 1))
    else:
        # child is leaf node
        lines.append(f"{prefix}{attr_name} = {value}: {child.cl}\n")
    return ".join(lines)

```

```

def classify_example(tree, example):

```

```

    # If leaf node, return its class
    if not tree.children:
        return tree.cl

    # If internal node, recurse on one child node
    value = example[tree.attr_index]
    # unseen (new) attribute value
    if value not in tree.children:
        return tree.cl
    return classify_example(tree.children[value], example)

```

```

def classify_dtstar(training_filename, testing_filename):

```

```

    # list of (list of attributes, class)
    train_data = load_data(training_filename, True)
    # list of list of attributes
    test_data = load_data(testing_filename, False)

    # build dt
    # assign index 0, 1, ..., n-1 to n attributes
    attr_indices = list(range(len(train_data[0][0])))
    tree = build_dt(train_data, attr_indices)

```

```
# print dt for report
# print(dt_to_string(tree))

# classify test data
results = []
for example in test_data:
    cl = classify_example(tree, example)
    results.append(cl)

return results
```